



VERZWEIGUNGEN

Coders.Eleven - JAVA

#1 if

Motivation

Aufbau/Ablauf

Logische Operatoren

#2 else if

Aufbau/Ablauf

Beispiel

#3 switch

Aufbau/Ablauf

Beispiel

#4 Programmierstil

Klammerstil

Bedingungsoperator



#1 if

- Motivation
- Aufbau/Ablauf
- Logische Operatoren





Motivation

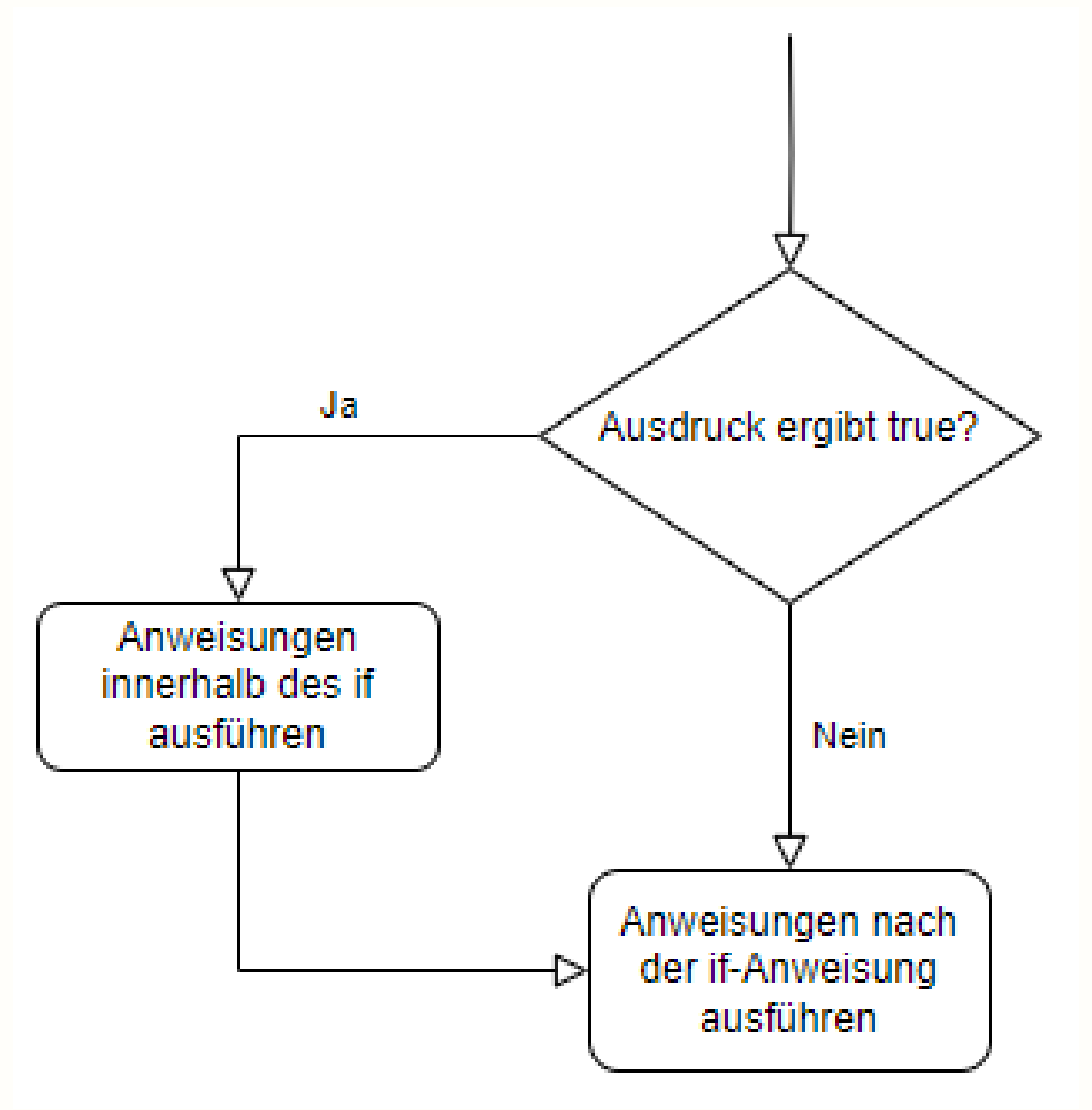
- Bisher wurden die einzelnen Zeilen eines Programms sequentiell abgearbeitet
- Sehr oft sollen aber bestimmte Anweisungen situationsabhängig ausgewählt werden
- Mit Verzweigungen kann der Ablauf des Programms situationsabhängig gesteuert werden

Aufbau

- Allgemeine Form

```
if (Ausdruck) {  
    Anweisungen  
}
```

- Ausdruck muss einen boolschen Wert (true/false) erzeugen
- Ablauf siehe Grafik



Beispiel

Die Methode `random` der Klasse `Math` liefert einen Wert (Zufallszahl) ≥ 0 aber kleiner als `1.0` (`double`) zurück

```
int first = (int) (Math.random() * 100);
int second = (int) (Math.random() * 100);

if(first > second) {
    System.out.println("first: " + first);
    System.out.println("second: " + second);
    System.out.println("first > second");
}

System.out.println("first: " + first);
System.out.println("second: " + second);
```

```
first: 34
second: 5
first > second
first: 34
second: 5
```

```
first: 70
second: 98
```


Komplexere Ausdrücke

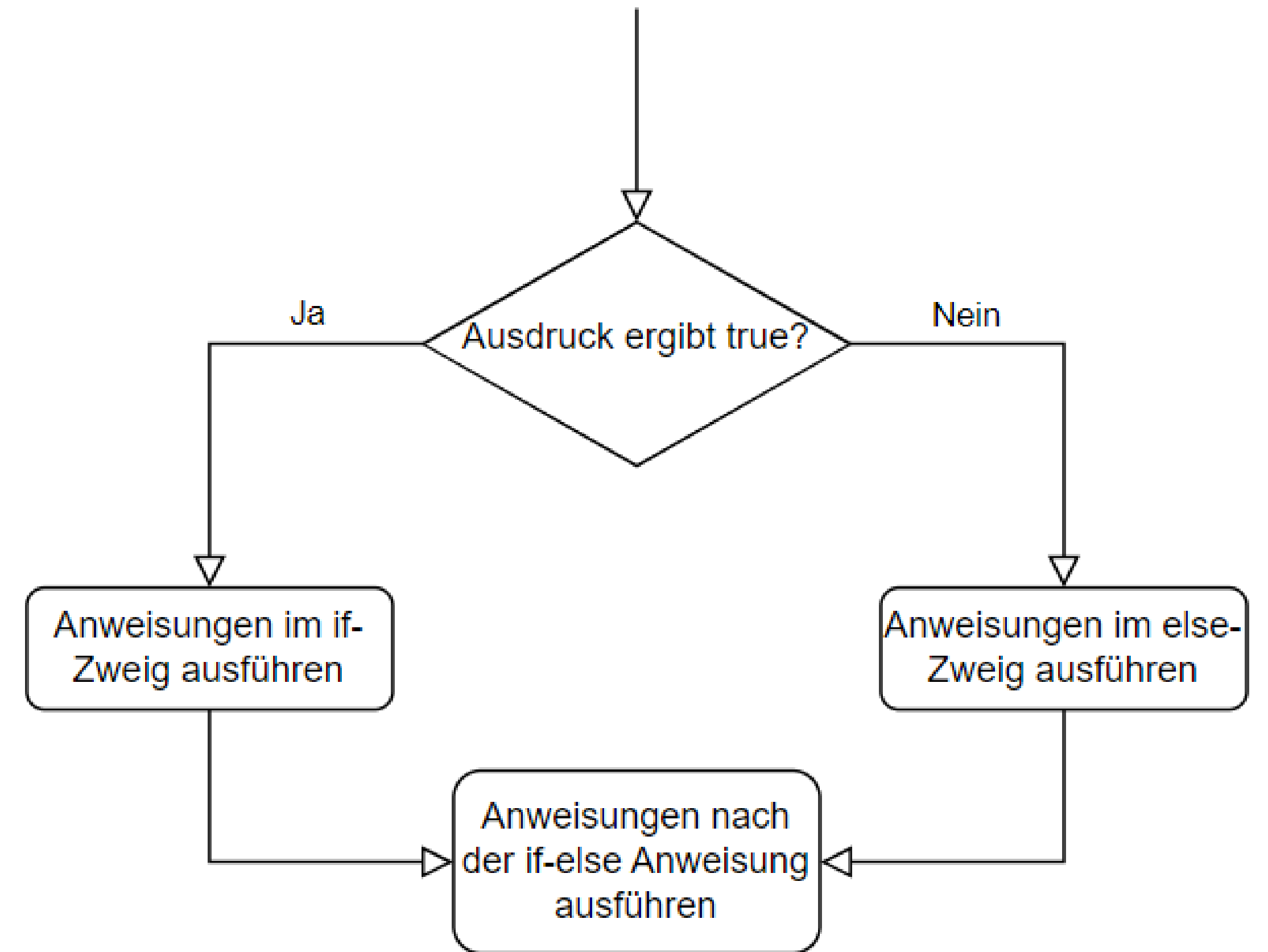
- Mehrere Vergleiche in einem Ausdruck verknüpfen
- Wird mit Hilfe von logischen Operatoren realisiert
- Teilweise Auswertung („Short circuit evaluation“)
 - `x && y` ist immer `false`, wenn `x == false`
 - Sobald `x` auf `false` ausgewertet, wird die gesamte Auswertung beendet
 - `x || y` ergibt immer `true`, falls `x == true`
 - Sobald `x` auf `true` ausgewertet, wird die gesamte Auswertung beendet
 - Achtung: in diesen Fällen kann `y` ungültige Werte enthalten (zB Division durch 0)
 - Vollständige Auswertung mit `&` und `|`

Aufbau

- Allgemeine Form

```
if (Ausdruck) {  
    Anweisungen im if-Zweig  
} else {  
    Anweisungen im else-Zweig  
}
```

- Ausdruck wie beim einfachen if
- Ablauf siehe Grafik



Beispiel

```
int first = (int) (Math.random() * 100);
int second = (int) (Math.random() * 100);

if(first > second) {
    System.out.println("first: " + first);
    System.out.println("second: " + second);
    System.out.println("first > second");
} else {
    System.out.println("first: " + first);
    System.out.println("second: " + second);
    System.out.println("first <= second");
}
```

```
first: 59
second: 31
first > second
```

```
first: 37
second: 81
first <= second
```

Beispiel (Maximum dreier Zahlen)

```
int firstNumber = (int) (Math.random() * 100);
int secondNumber = (int) (Math.random() * 100);
int thirdNumber = (int) (Math.random() * 100);

int maximum;

if(firstNumber > secondNumber) {
    if(firstNumber > thirdNumber) {
        maximum = firstNumber;
    } else {
        maximum = thirdNumber;
    }
} else {
    if(secondNumber > thirdNumber) {
        maximum = secondNumber;
    } else {
        maximum = thirdNumber;
    }
}
```

Diese if-Anweisungen sind unabhängig voneinander. Es kann eine beliebige Kombination davon ausgeführt werden

```
maximum = firstNumber;
if(secondNumber > maximum) {
    maximum = secondNumber;
}
if(thirdNumber > maximum) {
    maximum = thirdNumber;
}
```

Diese Zuweisung wird nur erreicht, falls $\text{firstNumber} > \text{secondNumber}$ und $\text{firstNumber} > \text{thirdNumber}$ gilt

#2 else if

Aufbau/Ablauf

Beispiel



Aufbau

- Allgemeine Form

```

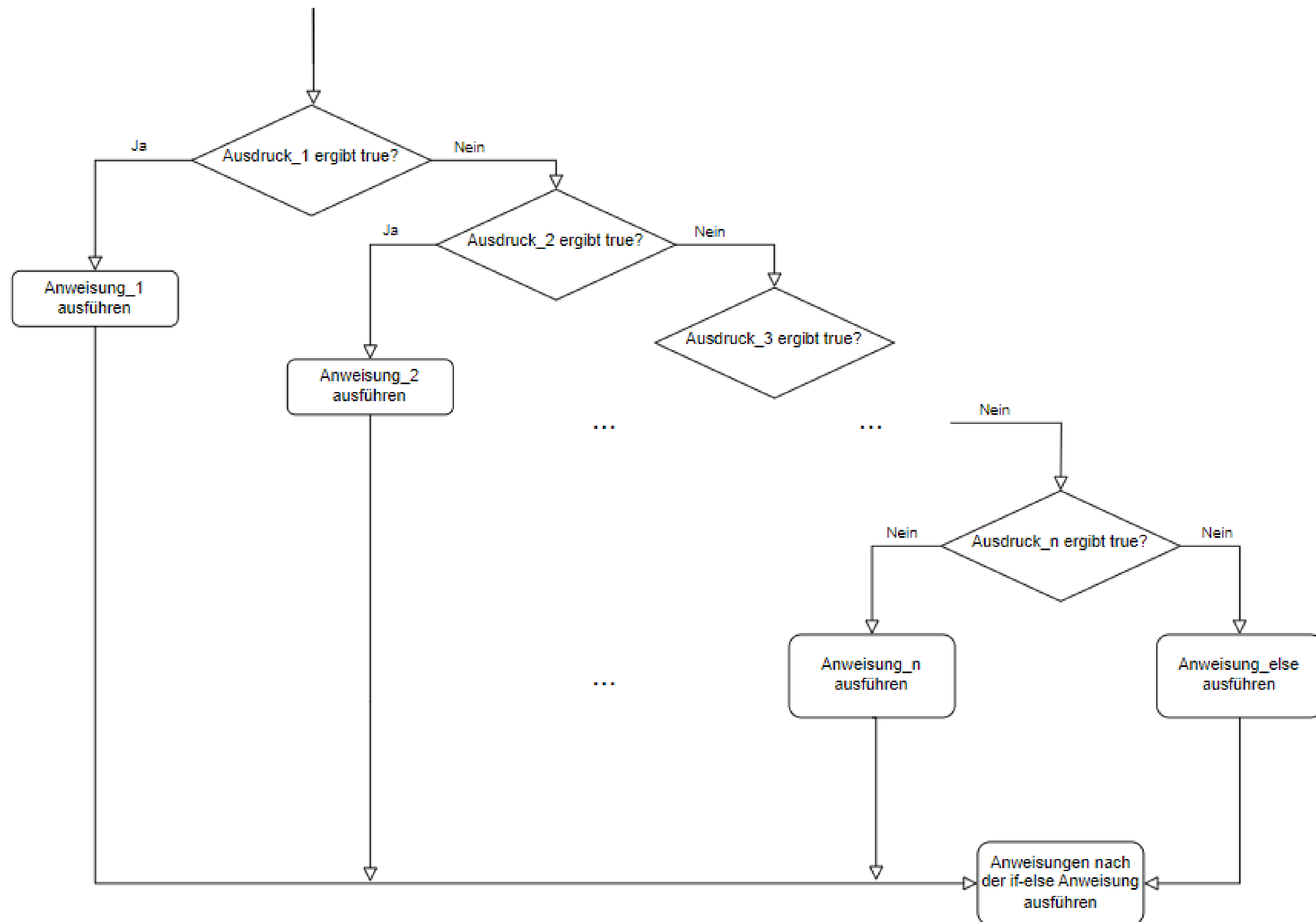
if (Ausdruck_1) {
    Anweisungen_1
} else if (Ausdruck_2) {
    Anweisungen_2
}
...
else if (Ausdruck_n) {
    Anweisung_n
} else {
    Anweisung_else
}

```

Ein Vergleich wird nach dem anderen durchgeführt

Bei der ersten Bedingung die wahr ist, wird die Anweisung abgearbeitet und dann die Anweisung nach der Verzweigung ausgeführt

• • • • Ablauf



Beispiel

```
Random random = new Random();
int day = random.nextInt( bound: 8);

if(day == 1) {
    System.out.println("Montag");
} else if(day == 2) {
    System.out.println("Dienstag");
} else if(day == 3) {
    System.out.println("Mittwoch");
} else if(day == 4) {
    System.out.println("Donnerstag");
} else if(day == 5) {
    System.out.println("Freitag");
} else if(day == 6) {
    System.out.println("Samstag");
} else if(day == 7) {
    System.out.println("Sonntag");
} else {
    System.out.println("Falsche Eingabe");
}
```


#3 switch

Aufbau/Ablauf

Beispiel



Aufbau

```
switch (Ausdruck) {
    case Wert_1:
        Anweisung_1
        break;
    ...
    case Wert_n;
        Anweisung_n
        break;
    default:
        Anweisung_default
        break;
}
```


• • • break

- Sorgt dafür, dass nicht zur folgenden case-Marke weitergesprungen wird
 - switch-Anweisung wird verlassen
- Ohne break können zwei oder mehrere case-Marken verbunden werden
 - Alle nachfolgenden Anweisungen werden bis zum nächsten break oder dem Ende der switch-Anweisung abgearbeitet
 - „Fall-through“

• • • default

- Ist optional
- Wird ausgeführt, wenn keine case-Marke passt
- Ohne default ist es möglich, dass keine Anweisungen ausgeführt werden
- Hinweis:
 - Jeder Wert darf nur einmal bei einer case-Marke vorkommen

Beispiel

```
switch(day) {  
    case 1:  
        System.out.println("Montag");  
        break;  
    case 2:  
        System.out.println("Dienstag");  
        break;  
    case 3:  
        System.out.println("Mittwoch");  
        break;  
    case 4:  
        System.out.println("Donnerstag");  
        break;  
    case 5:  
        System.out.println("Freitag");  
        break;  
    case 6:  
        System.out.println("Samstag");  
        break;  
    case 7:  
        System.out.println("Sonntag");  
        break;  
    default:  
        System.out.println("Falsche Eingabe");  
        break;  
}
```


„Neues“ switch

- Seit Java 14
- Syntaktische Änderung („- >“ statt „:“)
- Kein break mehr
 - Es existiert kein „Fall-through“
- switch kann einen Wert zurückgeben
 - Hilfsvariablen können vermieden werden
 - Geschieht mit dem Schlüsselwort `yield`
- Compiler überprüft auf Vollständigkeit
 - Alle möglichen Werte müssen abgedeckt werden (direkt oder mittels `default`)

Beispiel

```
switch (day) {  
    case 1 -> System.out.println("Montag");  
    case 2 -> System.out.println("Dienstag");  
    case 3 -> System.out.println("Mittwoch");  
    case 4 -> System.out.println("Donnerstag");  
    case 5 -> System.out.println("Freitag");  
    case 6 -> System.out.println("Samstag");  
    case 7 -> System.out.println("Sonntag");  
    default -> System.out.println("Falsche Eingabe");  
}
```



#4 Programmierstil

Klammerstil
Bedingungsoperator



Klammerstil

- OTBS – Stil („One True Brace Style”)
 - Standard in IntelliJ

```
if (firstNumber > secondNumber) {  
    if (firstNumber > thirdNumber) {  
        maximum = firstNumber;  
    } else {  
        maximum = thirdNumber;  
    }  
} else {  
    ...  
}
```

+ Mehr Code pro Zeile
+ Schließende Klamme steht
unter dem Konstrukt, zu dem
sie gehört

- Allman-Stil
 - Standard in Visual Studio

```
if (firstNumber > secondNumber)  
{  
    if (firstNumber > thirdNumber)  
    {  
        maximum = firstNumber;  
    }  
else  
{  
    maximum = thirdNumber; ...  
}
```

+ Blockbereiche gut erkennbar
- Mehr Zeilen pro Datei

OTBS-Stil

- Vollständige Klammerung
 - Auch bei einzeiligen Blöcken

```
if (a < 10)
    a = 10;
```

```
if(a < 10) {
    a = 10;
}
```

- Rechte Variante braucht mehr Platz, erleichtert aber die spätere Erweiterung

Verkürzungen

- Einfache `if`-Anweisungen werden manchmal einzeilig geschrieben

```
if(n != 0)    x = x/n;
```

- Nur sehr sparsam verwenden bzw. generell vermeiden

Anwendung von `if` und `switch`

- Abhängig von der Lesbarkeit und Ausdruckstärke
- `if`
 - Einfache Auswahl-situationen
 - Anweisungen ausführen oder nicht bzw. alternative Anweisungen ausführen
 - Komplexere Ausdrücke in Bedingungen
- `switch`
 - Anhand mehrerer konstanter Werte verzweigen

Bedingungsoperator

Beispiel für Verzweigung

```
int max, a, b;

...

if (a > b) {
    max = a;
} else {
    max = b;
}
```

Beispiel mit Bedingungsoperator

```
int a, b;

int max = (a > b) ? a : b;
```

- Nur sehr sparsam einsetzen